

Kafka MirrorMaker or Geo-Replication (Cross-Cluster Data Mirroring)

- ❖ Mirroring is replication between clusters – called ***mirroring*** to not confuse with ***replication***
- ❖ replication uses cluster involving brokers, partition leaders, partition followers, ISR and ZooKeeper
- ❖ mirroring is just a consumer/producer pair in two clusters
- ❖ ***MirrorMaker*** is used for replicating one clusters data to another cluster

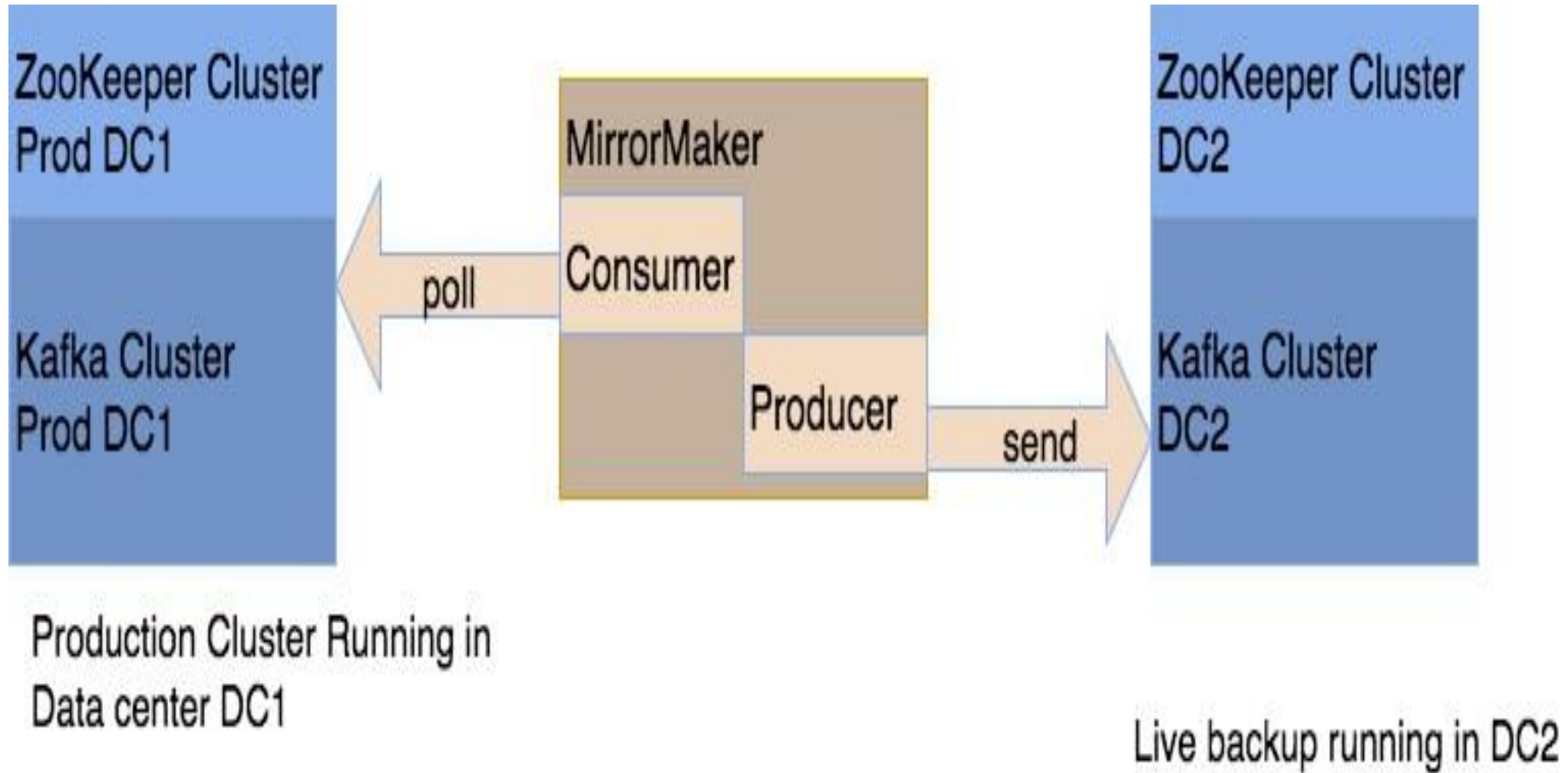
- ❖ ***MirrorMaker*** acts like a ***consumer*** to a ***source cluster***
- ❖ ***MirrorMaker*** acts like a ***producer*** to a ***destination cluster***
- ❖ ***Data read from source topics in source cluster and written to same named topics in destination cluster***
- ❖ MirrorMaker is built on top of the Kafka Connect framework
- ❖ Source and destination clusters are independent and not coupled
 - ❖ Topics can be configured differently, have different offsets
 - ❖ e.g., different partition count and different replication factors

MirrorMaker is built on top of the Kafka Connect framework and supports features such as:

- Replicates topics (data plus configurations)
- Replicates consumer groups including offsets to migrate applications between clusters
- Replicates ACLs
- Preserves partitioning
- Automatically detects new topics and partitions
- Provides a wide range of metrics, such as end-to-end replication latency across multiple data centers/clusters
- Fault-tolerant and horizontally scalable operations

- ❖ Provide a replica to another datacenter or AWS region
- ❖ Cloud migration or hybrid cloud deployments
- ❖ Mirroring used for disaster recovery
 - ❖ datacenter or region goes down
 - ❖ cluster is used for normal fault-tolerance
- ❖ Mirroring can also be used for increased throughput
 - ❖ scale consumers
 - ❖ scales reads

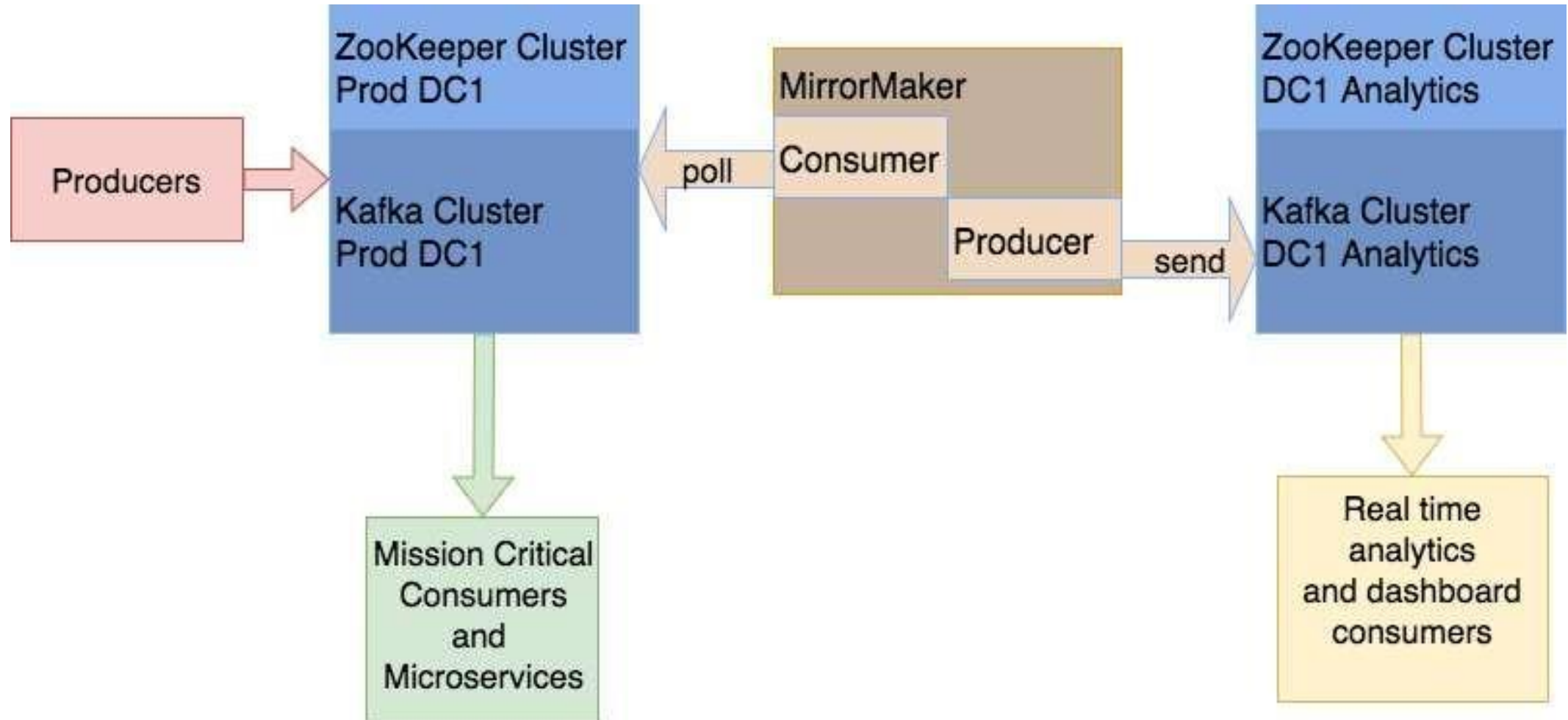
Disaster Recovery



- ❖ You can use MirrorMaker to scale reads
- ❖ You could move non-mission critical consumers to another cluster and replicate to this other cluster
- ❖ Other cluster can replay log or do read intensive log operations and analytics ***w/o impacting Production***
- ❖ Production cluster ***to serve mission critical services***
- ❖ Analytics cluster could be doing real time dash boards and analytics

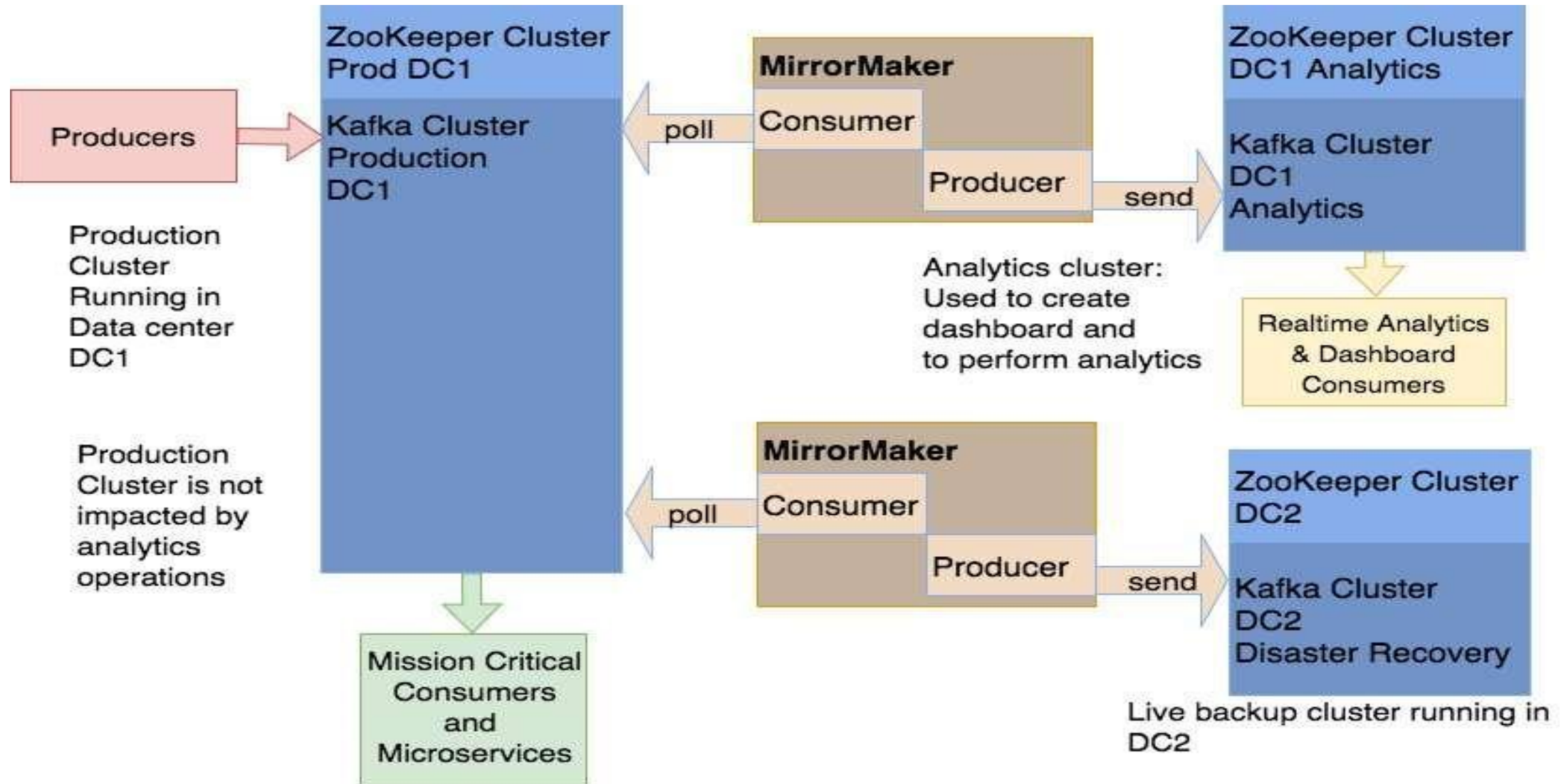
Scale Write, Avoid Impacting Mission Critical Services

Tos



Many MirrorMakers for different Purposes

Tos



With MirrorMaker, Kafka administrators can replicate topics, topic configurations, consumer groups and their offsets, and ACLs from one or more source Kafka clusters to one or more target Kafka clusters, i.e., across cluster environments. In a nutshell, MirrorMaker uses Connectors to consume from source clusters and produce to target clusters.

These directional flows from source to target clusters are called replication flows.

They are defined with the format `{source_cluster}->{target_cluster}` in the MirrorMaker configuration file as described later. Administrators can create complex replication topologies based on these flows.

- Active/Active high availability deployments: A->B, B->A
- Active/Passive or Active/Standby high availability deployments: A->B
- Aggregation (e.g., from many clusters to one): A->K, B->K, C->K
- Fan-out (e.g., from one to many clusters): K->A, K->B, K->C
- Forwarding: A->B, B->C, C->D

```
1 # Basic settings
2 clusters = primary, secondary
3 primary.bootstrap.servers = broker3-primary:9092
4 secondary.bootstrap.servers = broker5-secondary:9092
5
6 # Define replication flows
7 primary->secondary.enabled = true
8 primary->secondary.topics = foobar-topic, quux-.*
```

You can configure a variety of components in this file:

- MirrorMaker settings: global settings including cluster definitions (aliases), plus custom settings per replication flow
- Kafka Connect and connector settings
- Kafka producer, consumer, and admin client settings

By default as per [DefaultReplicationPolicy](#), the names of replicated topics in the target clusters have the format `{source}.{source_topic_name}`:

```
1 | us-west          us-east
2 | =====          =====
3 |                   bar-topic
4 | foo-topic  -->  us-west.foo-topic
```

To minimize latency ("producer lag"), it is recommended to locate MirrorMaker processes as close as possible to their target clusters, i.e., the clusters that it produces data to.

```
1 | First DC          Second DC
2 | =====          =====
3 | primary ----- MirrorMaker --> secondary
4 | (remote)                (local)
```

explicitly set these "local"
clusters in the
`--clusters` command line

```
# Run in secondary's data center, reading from the remote `primary` cluster  
$ ./bin/connect-mirror-maker.sh connect-mirror-maker.properties --clusters secondary
```

The **--clusters** secondary tells the MirrorMaker process that the given cluster(s) are nearby, and prevents it from replicating data or sending configuration to clusters at other, remote locations.

Example: Active/Active High Availability Deployment

```
1 # Bidirectional flow (two-way) between us-west and us-east clusters
2 clusters = us-west, us-east
3 us-west.bootstrap.servers = broker1-west:9092,broker2-west:9092
4 us-east.bootstrap.servers = broker3-east:9092,broker4-east:9092
5
6 us-west->us-east.enabled = true
7 us-east->us-west.enabled = true
```

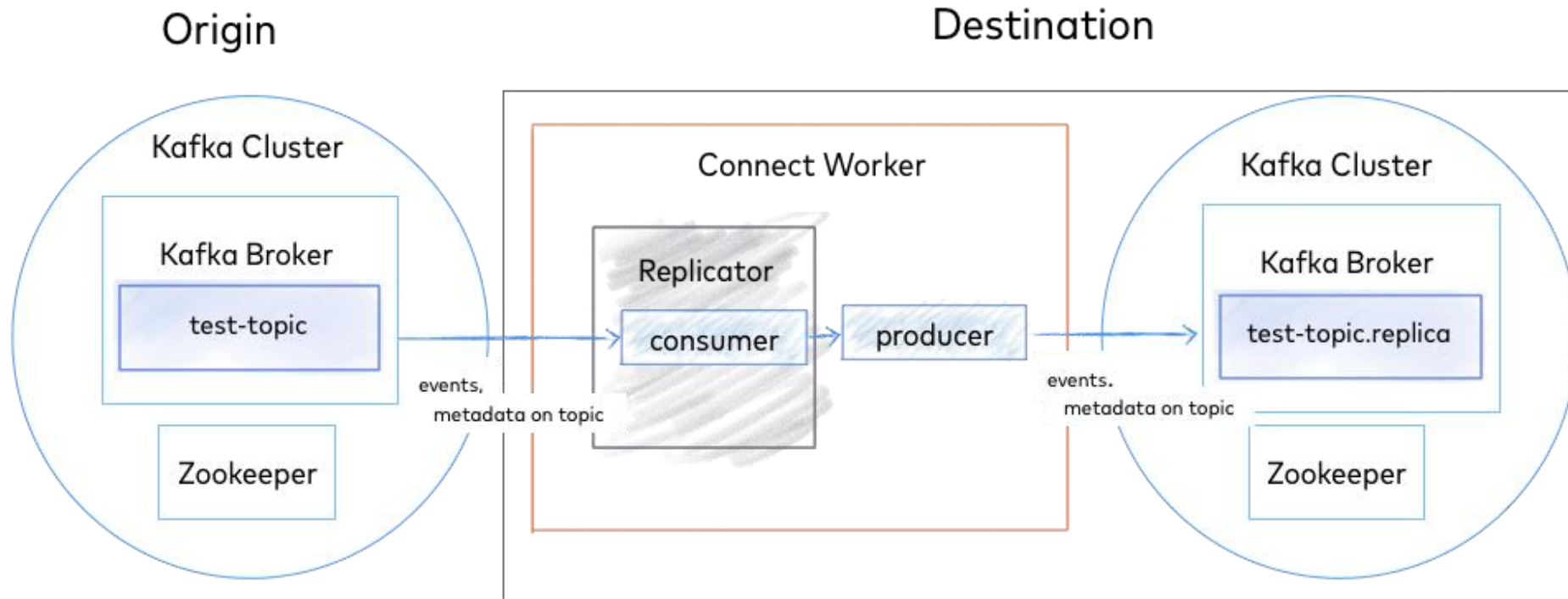
Note on preventing replication "loops" (where topics will be originally replicated from A to B, then the replicated topics will be replicated yet again from B to A, and so forth): As long as you define the above flows in the same MirrorMaker configuration file, you do not need to explicitly add **topics.exclude** settings to prevent replication loops between the two clusters.

Example: Multi-Cluster Geo-Replication

```
1  # Basic settings
2  clusters: west-1, west-2, east-1, east-2, north-1, north-2
3  west-1.bootstrap.servers = ...
4  west-2.bootstrap.servers = ...
5  east-1.bootstrap.servers = ...
6  east-2.bootstrap.servers = ...
7  north-1.bootstrap.servers = ...
8  north-2.bootstrap.servers = ...
9
10 # Replication flows for Active/Active in West DC
11 west-1->west-2.enabled = true
12 west-2->west-1.enabled = true
13
14 # Replication flows for Active/Active in East DC
15 east-1->east-2.enabled = true
16 east-2->east-1.enabled = true
17
18 # Replication flows for Active/Active in North DC
19 north-1->north-2.enabled = true
20 north-2->north-1.enabled = true
21
22 # Replication flows for XDCR via west-1, east-1, north-1
23 west-1->east-1.enabled = true
24 west-1->north-1.enabled = true
25 east-1->west-1.enabled = true
26 east-1->north-1.enabled = true
27 north-1->west-1.enabled = true
28 north-1->east-1.enabled = true
```

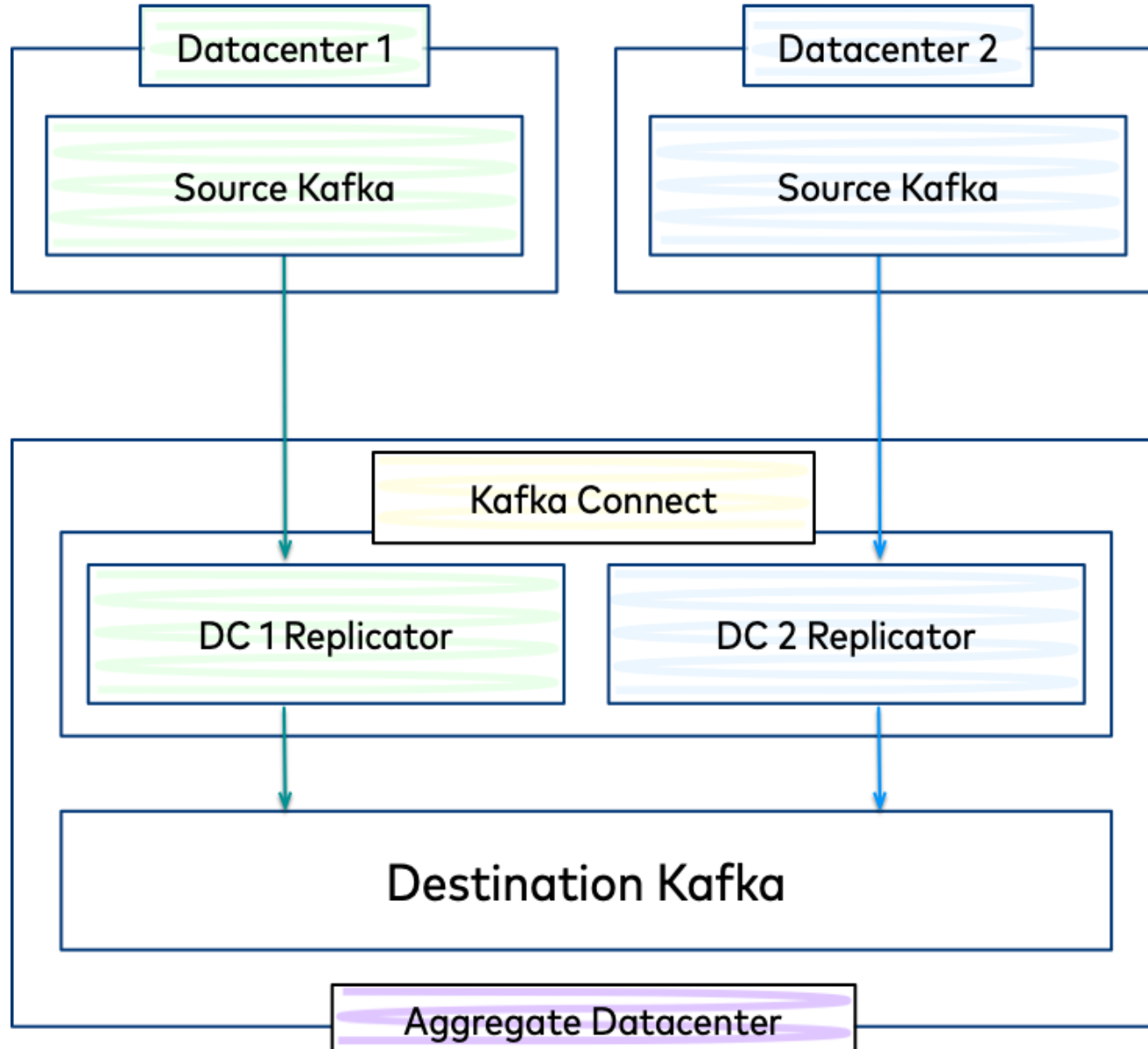
Confluent Replicator

Tos



Replicator Architecture

Confluent Replicator



Lab: Mirroring data between clusters – MirrorMaker V2 – 90 Minutes